Lecture 6 — NoSQL & Distributed Databases

CS4225 / CS5425 · Big Data Systems for Data Science · National University of Singapore

1 What is NoSQL?

NoSQL refers to **non-relational databases** — systems that store and retrieve data in formats other than the traditional row-and-column relational tables of SQL databases (RDBMS).

Etymology: "NoSQL" originally meant "No SQL" (no relational model). It has evolved to mean **"Not Only SQL"** — a pragmatic approach where relational and non-relational databases are used side-by-side, each for what it does best. Many modern NoSQL systems even support SQL-like query languages.

Traditional RDBMS (SQL)

Data in fixed tables with predefined schemas. Strong consistency (ACID). Great for complex queries and joins. Hard to scale horizontally. Examples: PostgreSQL, MySQL, Oracle.

NoSQL

Flexible schema, varied data models (key-value, document, column, graph, vector). Designed for horizontal scalability. Relaxed consistency. Examples: MongoDB, Redis, Cassandra.

Analogy: RDBMS is like a strict filing cabinet — everything must fit into labelled folders. NoSQL is like a flexible storage warehouse — you can store anything, organised however makes sense for your use case.

2 Why NoSQL? Motivation

The rise of the web and mobile applications created data challenges that traditional RDBMS were not designed to handle:

Volume

Social networks, e-commerce, IoT produce petabytes of data. RDBMS do not scale horizontally — adding machines does not proportionally increase capacity.

Velocity

High write/read rates (millions of requests per second). Requiring strict locking and ACID guarantees creates bottlenecks at scale.

Variety

Data is unstructured or semi-structured (JSON, images, graphs, vectors). RDBMS enforce rigid schemas that don't adapt well to changing or irregular data.

Key design properties that NoSQL systems adopt to address these challenges:

Horizontal scalability

Replicate and distribute data over many commodity servers (scale out, not up).

Simple call interface

Basic get/put operations (or simple queries) instead of complex SQL with joins.

Weaker consistency

Trade strict ACID for eventual consistency to gain availability and partition tolerance.

Distributed indexes & RAM

Efficient use of distributed in-memory indexes for low-latency lookups.

Flexible schemas

Different records in the same "collection" can have different fields — no migration needed for schema changes.

High availability

Replication across nodes ensures reads and writes continue even if some nodes fail.

3 Major Types of NoSQL Databases

Key-Value Stores

Simple lookup by key. Very fast. Like a giant hash map.

Document Stores

Store JSON-like documents. Queryable by field values.

Wide Column DBs

Rows can have different column sets. Great for sparse data.

Graph DBs

Nodes and edges. Optimised for relationship queries.

Vector DBs

Store high-dimensional vectors. Optimised for similarity search.

4 Key-Value Stores

Data model: A giant distributed hash map — each entry is a **(key, value)** pair. Keys are queryable primitives (strings, ints, bytes). Values can be anything — primitive, JSON, binary blob — but *cannot be queried by content*.

Core API — just two operations

`get(key)` → returns the value for that key

`put(key, value)` → stores / overwrites the value

Optional: `multi-get`, `multi-put`, `range queries`

When to use

Small, continuous reads and writes
Storing "basic" info (raw bytes, session data, user preferences)
When complex queries (joins, filters on value content) are rarely needed

Not suitable when you need: "find all patients who took treatment X"

Non-persistent **In-memory hash table**

All data lives in RAM for ultra-low latency. Data may be lost on restart (or backed up periodically to disk).

Examples: **Memcached**, **Redis**

Persistent **Disk-backed store**

Data survives restarts. Typically uses LSM-trees or B-trees on disk.

Examples: **RocksDB**, **Dynamo** (Amazon), **Riak**

Origin: Key-value stores are based on Amazon's landmark *Dynamo* paper (2007), which introduced many techniques still used today (consistent hashing, vector clocks, quorum reads/writes).

5 Document Stores

Document stores extend key-value stores by giving structure to values — each value is a **document** (a JSON-like object with fields and nested sub-objects). Because documents have structure, the database can **query by field content**, not just by key.

Data model hierarchy:
Database → multiple **Collections** → each collection has multiple **Documents** → each document has **Fields & Values**

Key property: *different documents in the same collection can have different fields* (schema-free / dynamic schema). Values can be nested JSON objects.

Example document

```
{
 "_id": "u123",
 "name": "Alice",
 "email": "alice@example.com",
 "address": { "city": "Singapore", "zip":
"119077" },
 "tags": ["student", "cs4225"]
}
```

Querying — CRUD operations

Create: insert a new document
Read: query by field value, e.g. `find({'class':'cs4225'})`
Update: modify specific fields of matching documents
Delete: remove documents matching a filter

This is a major advantage over key-value stores — you can filter by document content.

Advantages

Flexible schema handles evolving data. Rich querying on fields. Natural fit for JSON-heavy web apps.

Popular systems

MongoDB (most widely used), CouchDB, Firebase Firestore. MongoDB is consistently ranked among the most popular databases globally.

Key distinction: Key-value stores cannot query *inside* the value. Document stores can filter, sort, and index on any field *within* the document.

6 Other NoSQL Types: Wide Column, Graph, Vector

Wide Column Databases

Rows can have different sets of columns — great for sparse data where most cells would be NULL in a traditional table.

Organised by column families. Highly efficient for "fetch all data for user X" queries.

Examples: **Apache Cassandra**, **HBase**, **Google Bigtable**

Graph Databases

Data stored as **nodes** (entities) and **edges** (relationships). Optimised for traversing relationships — "friends of friends", shortest path, community detection.

Very inefficient in RDBMS (join explosion). Native graph storage makes these queries fast.

Examples: **Neo4j**, **Amazon Neptune**

Vector Databases

Store high-dimensional dense vectors (embeddings from AI/ML models). Core operation: **approximate nearest neighbour (ANN) search** — find the most similar vectors to a query.

Related to LSH / min-hashing from Lecture 5. Critical for AI search and recommendation.

Examples: **Milvus**, **Weaviate**, **Redis**, **MongoDB Atlas**

Vector DBs and AI: Large Language Models convert text to vector embeddings. Vector databases enable fast similarity search over billions of these embeddings — powering semantic search, recommendation systems, and RAG (retrieval-augmented generation) in modern AI applications.

6b Vector Databases — Deep Dive

A vector database stores data as **lists of numbers** (called **vectors** or **embeddings**), and its superpower is answering one question: *"which stored items are most similar to this query?"*

The core problem it solves: Traditional databases answer exact questions — `WHERE name = 'Alice'`. But many modern problems are about *meaning and similarity*: "find songs that sound like this", "find documents that answer this question", "find images that look like this photo". None of these can be solved with a simple equality check. They need **similarity search**.

Step 1 — Convert data into vectors (embeddings)

An AI model takes raw data (text, images, audio) and converts it into a fixed-length list of numbers that captures *meaning*. Things with similar meaning get similar vectors:

```
"The cat sat on the mat"   →  [0.21, -0.85, 0.44, 0.03, ...]   ← 768 numbers
"A feline rested on a rug" →  [0.19, -0.82, 0.41, 0.07, ...]   ← very similar!
"Stock prices rose today"  →  [-0.60, 0.12, -0.33, 0.91, ...]  ← very different
```

The first two sentences have *similar* vectors because they mean similar things, even though they share almost no words.

Step 2 — Store vectors in the database

Millions of these vectors are indexed. Each one is a point in very high-dimensional space (e.g. 768 dimensions).

Step 3 — Query: find the nearest neighbours

The user's query is also converted to a vector. The database finds stored vectors **closest** to the query vector — these are the most semantically similar results.

```
Query: "feline on a rug"  →  query vector: [0.20, -0.83, 0.42, 0.06, ...]
Closest stored vector:       "A feline rested on a rug"  ✓  (returned as top result)
```

How "closeness" is measured: Usually **cosine similarity** (the angle between two vectors) or Euclidean distance. Vectors pointing in the same direction = similar meaning.

Algorithm used: Approximate Nearest Neighbour (ANN) search — and specifically **Locality-Sensitive Hashing (LSH)**, which you covered in Lecture 5. Rather than comparing the query to every single stored vector (impossibly slow at scale), LSH groups similar vectors into the same "buckets" so you only compare against a small candidate set. You get a very good answer very fast — not necessarily the mathematically perfect one. That's the "approximate" part.

Analogy — the smart librarian

A traditional database finds books *by exact title or ISBN*. A vector database is like a librarian who has read every book and can say: "You liked this novel — here are 5 others with a very similar tone, theme, and writing style," even if none share a word in the title. The AI model does the "reading"; the vector database does the "recommending" at scale.

Why a full database, not just an algorithm?

The ANN algorithm alone doesn't give you:
- **Scalability** — billions of vectors across many machines
- **Real-time updates** — new data without rebuilding the entire index
- **Replication** — copies for fault tolerance
- **Filtering** — "find similar products that are also in stock under $50"

A vector database packages all of this together.

Real-world applications:

Application	What is stored as vectors	The query
Semantic search	Web page / document text embeddings	Search query → find similar-meaning pages
Music recommendation	Song audio embeddings	Listening history → find similar-sounding songs
LLM memory (RAG)	Chunks of documents / knowledge base	User question → find most relevant passages
Reverse image search	Image embeddings from vision model	Upload photo → find visually similar images

Exam tip: The lecture explicitly connects vector databases back to **LSH / min-hashing from Lecture 5**. The likely exam angle is: what problem do vector databases solve, how do they relate to LSH, and why are they important for AI applications? You won't need to implement ANN — understand the concept, motivation, and the LSH link.

7 Exam-Style Q&A

Q1 (from slides): True or false — an advantage of document stores over key-value stores is that it is easier to store unstructured objects.

> **False.** Key-value stores can also store unstructured values (raw bytes, blobs, JSON). The real advantage of document stores is that they allow *querying by the content of the value* (e.g. find all documents where class = 'cs4225'). Key-value stores can only look up by key.

Q2 (from slides): True or false — an advantage of document stores over key-value stores is that it is easier to query for all documents fitting a condition.

> **True.** Document stores index document fields and support content-based queries (CRUD with filter conditions). Key-value stores only support exact-key lookup — you cannot ask "give me all values where the embedded 'class' field = 'cs4225'".

Q3 (from slides): True or false — in MongoDB, shards are beneficial to allow for redundancy.

> **False.** In MongoDB, *sharding* is for horizontal partitioning — distributing data across nodes to increase capacity. *Redundancy* (fault tolerance) is provided by *replica sets* (each shard having multiple identical copies). The two mechanisms serve different purposes.

Q: What is the difference between the "C" in ACID and "Strong Consistency" in distributed systems?

> Completely different. ACID Consistency = the database obeys predefined integrity constraints (e.g. balances can't go negative). Strong Consistency (distributed) = all nodes see the same value immediately after a write. These terms come from different communities and have been a persistent source of confusion.

Q: What is eventual consistency and when is it dangerous?

> Eventual consistency means that if the system is functioning and enough time passes, all replicas will converge to the last written value. In the short term, stale reads are possible. It is dangerous when correctness matters immediately — e.g. bank balances, inventory counts, medical dosing. It is acceptable for social feeds, analytics, likes, and views.

Q: What is partition pruning in MongoDB, and when does it NOT apply?

> Partition pruning = the mongos router consults the config server to determine which shard holds the data, and only queries that shard. This applies when the query filter includes the *shard key*. If the query filters on a *non-shard key* field, the router doesn't know which shard has matching data, so it must send the query to all shards (scatter-gather), which is less efficient.

Q: What is the main risk of denormalization?

> Update anomalies. If duplicated data changes (e.g. a user renames themselves), every document containing that duplicated value must be updated. This is a write-heavy operation that grows with how many copies exist. Normalisation avoids this — update once, join on read — but joins are expensive at scale.

SUPPLEMENTARY

8 Consistency Models: Strong vs Eventual

In a distributed system, data is replicated across multiple nodes. When a write happens, those copies need to be updated. The question is: *how quickly must all nodes agree?*

Strong Consistency

Any read *immediately after* a write returns the new value — on **all** nodes, from **all** observers, at the same time.

Imagine a weighing scale: if you update your weight to 60kg, every device reading that value *immediately* shows 60kg — no exceptions.

Pros Simple to reason about; no stale reads
Cons Requires coordination between nodes; higher latency; availability may suffer

Eventual Consistency

If the system is functioning and you wait long enough, all nodes will *eventually* converge to the last written value. In the short term, some nodes may return stale data.

Same weighing scale: right after the update, some devices may still show 50kg (old value). But eventually, the update propagates and all show 60kg.

Pros Higher availability; lower latency; survives network partitions
Cons Application must handle stale reads; complex edge cases

When is eventual consistency acceptable?
OK Social media feeds, tweet counts, product views, statistical queries — slight staleness doesn't matter
NOT OK Bank transfers, inventory (can't oversell), medical records — stale data causes real harm

Tunable consistency: Many modern NoSQL systems (e.g. Cassandra, MongoDB) allow you to *configure* the consistency level per-operation — you can choose "read from majority of replicas" (strong) vs "read from any replica" (eventual). This is called **tunable consistency**.

9 ACID vs BASE

These are two philosophies for guaranteeing data correctness in database systems:

ACID (traditional RDBMS)

Atomicity: a transaction is all-or-nothing
Consistency: predefined constraints always hold (e.g. balance ≥ 0)
Isolation: concurrent transactions don't interfere
Durability: committed data survives crashes

Note: the "C" in ACID is different from strong/eventual consistency in distributed systems — don't confuse them!

BASE (many NoSQL systems)

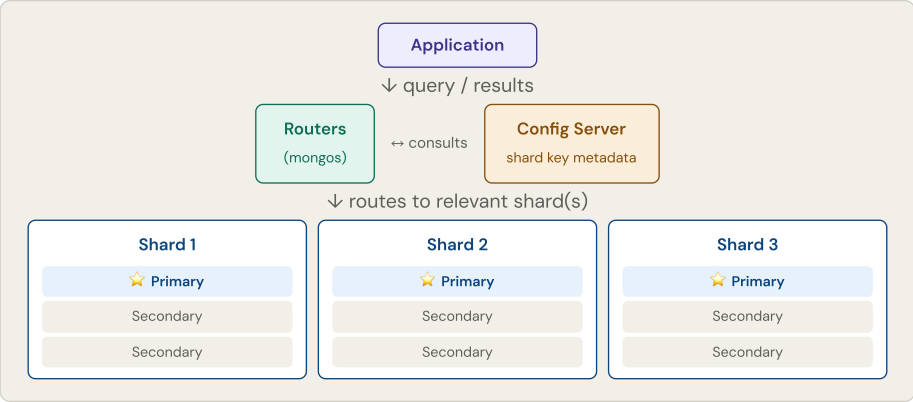
Basically **A**vailable: reading and writing are available most of the time
Soft State: without strict guarantees, the state may be uncertain at any moment
Eventually Consistent: the system will converge to consistency given enough time

BASE trades ACID's strict guarantees for better scalability and availability.

Exam trap — ACID "C" vs distributed "Consistency": In ACID, Consistency means the database obeys integrity constraints (like no negative balances). In distributed systems, Strong Consistency means all nodes see the same data simultaneously after a write. These are **completely different concepts** from different communities. The exam will test whether you know this distinction.

10 MongoDB Architecture — Sharding & Replication

MongoDB is the most popular document store. It scales horizontally through two mechanisms: **sharding** (distributing data) and **replication** (duplicating data for fault tolerance).



Routers (mongos)

Accept queries from the application. Consult the config server to determine which shard(s) hold the relevant data, then route the query accordingly. Merge results before returning to the application.

Config Server

Stores **metadata**: which ranges of the shard key map to which shard. The router uses this to decide where to send each query — enabling **partition pruning** (only querying the relevant shard).

Shards (Replica Sets)

Each shard is a **replica set**: 1 primary + 2 secondaries holding the same data subset. A shard is responsible for a horizontal partition (range of the shard key) of the database.

MongoDB splits data by a **shard key** — a field you choose (e.g. `user_id`). Every document is assigned to exactly one shard based on its shard key value. The config server tracks which ranges of the shard key live on which shard:

```
Shard 1:  user_id  0         -  999,999
Shard 2:  user_id  1,000,000 - 1,999,999
Shard 3:  user_id  2,000,000 - 2,999,999
```

When a query includes the shard key (e.g. `find({user_id: 500000})`), the router knows it only needs to check Shard 1 — this is **partition pruning**. If the query uses a different field, all shards must be checked.

End-to-end query flow for `find({'class':'cs4225'})`:

1. Application sends `find({'class':'cs4225'})` to a **mongos router**.
2. mongos consults the **config server**. It checks: is `class` the shard key? No — the shard key is `user_id`. So the config server cannot narrow it down to one shard.
3. Because `class` is a *non-shard key* field, mongos must send the query to **all three shards** (scatter-gather). If instead the query were `find({user_id: 500000})`, the config server would know only Shard 1 holds that range — **partition pruning** — and only Shard 1 would be queried.
4. Each shard searches its own documents for `class = 'cs4225'` and returns its matches: e.g. Shard 1 returns `{name:'alice', class:'cs4225'}`, Shard 2 returns `{name:'bob', class:'cs4225'}`.
5. mongos **merges** the results from all shards into one list and returns it to the application: `[{name:'alice',...}, {name:'bob',...}]`.

Replication within a shard:

Writes

All writes go to the **Primary**. The primary records operations in an *operation log (oplog)*. Secondaries replicate the oplog and apply it to their local data copy, then acknowledge completion.

Reads & Consistency

By default, reads can come from **secondaries** (lower latency, more throughput, but may read stale data = *eventual consistency*). Configured to read only from **primary** = strong consistency but higher latency.

Shards vs Replica Sets — don't confuse:
Sharding = dividing the dataset across multiple shards (horizontal partitioning) → improves *capacity and throughput*
Replication = each shard is a replica set with multiple identical copies → improves *fault tolerance and availability*

11 Duplication (Denormalization)

In relational databases, data is **normalised** — stored in separate tables and joined at query time. NoSQL systems often instead use **denormalization**: embedding related data directly into the same document, even if it means duplicating it.

Normalized (RDBMS approach)

Users table: `user_id | name`
Posts table: `post_id | post | user_id`

To show posts with usernames, you must JOIN the two tables — expensive at scale.

Denormalized (NoSQL approach)

Posts table: `post_id | post | user_id | name`

The username is duplicated into every post record. Reading a post gives you everything you need — no join required. Fast, but redundant.

Why denormalize? "Storage is cheap — why not duplicate data to improve query efficiency?"

Tables are designed around the *queries we expect*. If 99% of reads need post + username together, pre-joining avoids expensive lookups at scale.

Problem with denormalization: If a user changes their name, that update must be propagated to *every* document that contains it. This is called an **update anomaly** and is the main cost of denormalization. You trade write complexity for read speed.

12 NoSQL vs RDBMS — Trade-offs

Dimension	NoSQL	RDBMS (SQL)
Schema	+ Flexible / dynamic — different records can have different fields	Rigid predefined schema — all rows must conform
Scalability	+ Horizontal (scale out) — add commodity servers	Mostly vertical (scale up) — harder to distribute
Availability	+ High availability via replication and relaxed consistency	Lower availability; strict locking reduces throughput
Query language	– No universal declarative query language; joins handled in application code	Powerful SQL with joins, aggregations, transactions
Consistency	– Weaker guarantees (eventual consistency); stale reads possible	Strong ACID guarantees; always consistent
Best for	Web apps, user sessions, social feeds, IoT, ML embeddings, unstructured data	Financial transactions, inventory, anything requiring strict correctness

The verdict is not binary. The right choice depends entirely on the application:
- Is denormalization acceptable (duplicating data)?
- How complex are the queries — simple lookups or multi-table joins?
- How critical is consistency — financial transactions or social media likes?
- What is the data volume and required throughput?

Many production systems use **both**: RDBMS for critical transactional data, NoSQL for high-volume/low-latency access patterns.